Energy and Resource Profiling of OPT-350m Training

COMP 597 – Sustainability in Systems Design
McGill University, Winter 2026

Aly Mohamed

April 9 2026

Abstract

We profile training of OPT-350m across batch sizes 4, 8, and 16 on a shared Slurm GPU cluster, measuring wall-clock time, energy consumption, and per-step resource usage under three instrumentation profiles. Fine-grained profiling adds less than 0.1% overhead in all configurations, confirming that detailed measurement is practically free for this workload. GPU utilization never reaches 100%, ranging from 83% at batch size 4 to 92% at batch size 16. The optimizer step consumes a constant 96 ms regardless of batch size, becoming the dominant source of underutilization at small batches where it accounts for 39% of step time. Larger batches process more samples per energy unit but require proportionally more energy per step. The total run energy stays nearly flat across a fixed 5-minute window.

1 Introduction

Training large language models is increasingly expensive in both compute time and energy. Understanding where that energy goes at a fine-grained level is a first step toward reducing it. This project builds a controlled measurement framework around Meta's OPT-350m model and uses it to answer three questions: (1) how much overhead does instrumentation add, (2) how does batch size affect throughput and energy efficiency, and (3) where inside the training loop does GPU utilization fall short of the theoretical maximum?

We run fixed-duration experiments (300 seconds each) under three profiling configurations and sweep batch sizes 4, 8, and 16, repeating each configuration three times to estimate variance.

2 Experimental Setting

Workload and Model. We evaluate the OPT synthetic training workload using `facebook/opt-350m` in full precision (FP32), sequence length 1024, learning rate 10^{-6}, and fixed data-loader workers (`num_workers=0`). A **step** is defined as one parameter update composed of three **phases**: (i) forward pass, (ii) backward pass, and (iii) optimizer update. Table 1 lists the fixed hyperparameters used across all runs.

Table 1: Fixed hyperparameters for all experiments.

Parameter	Value
`hf_name`	`facebook/opt-350m`
`dtype`	`fp32`
`seq_len`	1024
`learning_rate`	1×10^{-6}
`num_workers`	0
`max_samples`	10^8
`seed`	1234
`warmup_steps`	1
`max_duration_sec`	300.0

Runtime Protocol. Each final experiment is time-bounded to 5 minutes (300 s) using wall-clock stopping (`max_duration_sec=300`). For every configuration, we run **3 repeats** with the same seed and aggregate results as mean and standard deviation across repeats.

Batch-Size Sweep. We follow a power-of-two batch sweep protocol: (1) probe the maximum feasible batch size with OOM-safe backoff, (2) round down to the nearest power of two, and (3) evaluate three descending powers of two $[B_{\max}, B_{\max}/2, B_{\max}/4]$. For this model and hardware, the evaluated batch sizes are $16, 8, 4$. We have had GPU memory issues for batch sizes larger than 16. If you look at the results below in Figure 4, you will see that for batch size 16, our memory utilization is close to 100%.

Measurement Profiles. We separate instrumentation into three profiles:

- **Profile A (baseline):** end-to-end runtime only (`trainer_stats=noop`).
- **Profile B (energy):** end-to-end GPU energy and resource logging with reduced overhead.
- **Profile C (fine-grained):** phase timing (`simple`) combined with resource and energy logging (`resource`) for full timeline analysis.

Resource logging is throttled to ≈500 ms via `sample_interval_ms=500`, and flush frequency is reduced via `flush_every_n` to limit logging overhead.

Collected Metrics. From the three profiles, we collect: end-to-end duration per run per-step phase times with CUDA synchronization boundaries. Timeline metrics covering GPU utilization, GPU memory, process CPU utilization, and GPU power and energy . Post-processing aligns measurements by elapsed second, then reports mean ± std across 3 repeats.

Reporting and Validation. We generate per-batch timelines with variance bands, phase-time summaries, cross-batch comparisons of throughput and energy, and instrumentation overhead relative to Profile A. Protocol checks verify: (i) 3 repeats per batch/profile, (ii) run near 300 s within tolerance, and (iii) resource logging cadence consistency. GPU idle windows (intervals where utilization falls below a threshold) are also detected to identify efficiency opportunities.

3 Methodology

3.1 Training Loop

Each training step consists of three phases in sequence: a forward pass, a backward pass via `loss.backward()`, and an optimizer update via AdamW. CUDA is asynchronous, so a `torch.cuda.synchronize()` call is inserted before and after each phase timer. Without this, the timer stops when the CPU finishes dispatching work to the GPU, not when the GPU actually finishes it, leading to wildly optimistic phase durations.

The outer loop is fixed-duration rather than epoch-based: it exits when elapsed time exceeds `max_duration_sec = 300`. Warmup period is configured to 1 step, all steps from the second are included in measurement.

3.2 Measurement Stack

CodeCarbon v3.1.0 samples GPU power via NVML internally at approximately 500 ms intervals, integrating over time to produce energy in kWh and carbon in gCO_2e. We run in offline mode with a fixed carbon intensity of 40 gCO_2/kWh corresponding to the Quebec grid. Three tracker instances are created per run: one for the full session, one for individual steps, and one for individual phases (substeps).

nvidia-ml-py v13.580.82 queries NVML hardware counters directly at each step, reading `gpu_power_w` (instantaneous Watts), `gpu_energy_mj` (millijoules from the hardware counter), `gpu_util` (%), and `gpu_mem_used` (MiB). This gives step-level energy granularity without relying on CodeCarbon's sampling schedule.

psutil v7.1.3 collects system and process CPU utilization, RAM usage (RSS and VMS), and disk I/O counters at each step.

3.3 Reproducibility

All runs use a fixed random seed (1234), and the run configurations are all saved to `manifest.json` for every run. The full CLI invocation is recorded in the manifest; Listing 1 shows a representative command for batch size 8, Profile A.

Listing 1: Representative run command (batch size 8, Profile A, repeat 2).

```
./scripts/srun.sh
  --model opt --data opt --trainer simple
  --learning_rate 1e-06 --seed 1234
  --model_configs.opt.hf_name facebook/opt-350m
  --model_configs.opt.dtype fp32
  --data_configs.opt.batch_size 8
  --data_configs.opt.seq_len 1024
  --data_configs.opt.num_workers 0
  --data_configs.opt.max_samples 100000000
  --trainer_configs.simple.max_duration_sec 300.0
  --trainer_configs.simple.warmup_steps 1
  --trainer_stats noop
```

An overhead compliance threshold of 5% is applied: any profile adding more than 5% to the baseline duration would be considered too expensive to use routinely.

4 Results

4.1 Measurement Overhead

Table 2 shows the overhead of Profiles B and C relative to Profile A. Our energy profiling method adds 1.46–1.61% across all batch sizes, consistent with the polling cost of the NVML-based sampler. All values are well within the 5% compliance threshold.

Table 2: Profiling overhead by batch size.

	Batch 4	Batch 8	Batch 16
Profiling overhead	1.49%	1.46%	1.61%

4.2 Throughput and Energy Efficiency

Figure 1 summarizes how batch size affects throughput and energy. Steps per second decrease with

batch size (notice 4.05 at batch 4, and 1.58 at batch 16) because each step processes more data and takes longer. Samples per second, however, increase from 16.2 to 25.3, meaning larger batches deliver more useful work per unit time.

Energy per step grows roughly in proportion to batch size (12.0, 19.3, and 35.3 μWh), but total run energy stays nearly flat (14.6, 15.9, and 16.7 mWh) because the window is fixed at 300 seconds. The energy efficiency metric, samples per joule, rises from 0.038 at batch 4 to 0.059 at batch 16, a 55% improvement. Larger batches are more energy-efficient in absolute terms.

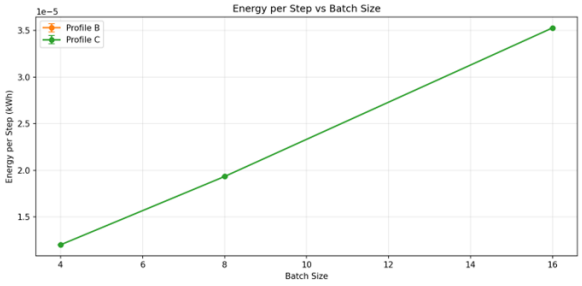


Figure 1: Throughput (samples/sec), energy per step, and energy efficiency (samples/J) across batch sizes. Each bar is the mean over three 5-minute runs.

4.3 Phase Timing

Figure 2 shows per-step phase durations and their share of total step time. Forward and backward times scale approximately linearly with batch size: forward goes from 43.0 to 180.3 ms, and backward from 105.3 to 352.1 ms (roughly 2× the forward time at each batch size).

The optimizer step is a clear outlier: it remains at 96 ± 0.2 ms across all three batch sizes. This is expected because the AdamW update operates over the model's parameter tensors, whose size is fixed at 350M parameters regardless of how much data was processed in the forward pass. At batch 4 the optimizer accounts for 39% of step time; at batch 16 this drops to 15% while the backward pass grows to 56%.

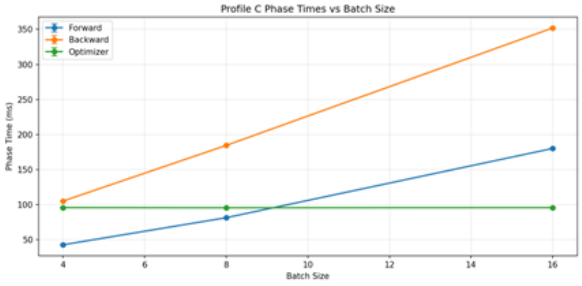


Figure 2: Left: share of step time for each phase. Right: absolute phase durations in ms. The optimizer duration is constant across batch sizes.

4.4 GPU Utilization

Figure 3 shows average GPU utilization and the number of detected low-utilization windows per run. GPU utilization never reaches 100% in any configuration. Batch 4 averages 83.3%, with one continuous low-utilization window spanning the entire run. Batches 8 and 16 average approximately 92% but show periodic dips: 3 windows for batch 8 and 29 for batch 16.

The pattern at batch 4 is consistent with the phase timing data: when the optimizer takes 39% of step time but does not fully saturate the GPU (the AdamW update launches many small CUDA kernels and is partly memory-bandwidth-bound), average utilization drops. At larger batches the forward and backward passes dominate, but the optimizer still creates brief recurring dips.

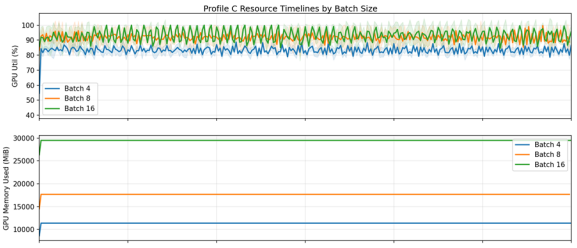


Figure 3: Left: average GPU utilization per batch size. Right: number of low-utilization windows (below 95%) detected per run.

The full resource timelines in Appendix A show GPU utilization, memory, CPU, and cumulative energy over time for each batch size. GPU memory scales as expected with batch size, and system CPU stays low throughout, confirming that the synthetic data loader imposes no I/O overhead.

5 Optimization Opportunities

The profiling results point to a few concrete places where efficiency can be improved.

Fused optimizer kernels. The 96 ms AdamW step is the clearest target. It costs the same regardless of batch size, which means at batch 4 it is eating 39% of every step while the forward and backward passes are relatively short. The root cause is that the standard AdamW implementation launches a large number of small CUDA kernels, one per parameter tensor. Each of which incurs its own launch overhead and competes for memory bandwidth without fully saturating it. Switching to a fused implementation collapses these into a single kernel, which should meaningfully reduce this phase. It would not be surprising to see the optimizer time drop by half or more, which at batch 4 would recover most of the underutilization observed there.

Larger batches via gradient checkpointing. Going from batch 4 to batch 16 improved energy efficiency by 55% on a samples-per-joule basis, and the curve was not yet flat. The reason we stopped at 16 is GPU memory, not compute. Gradient checkpointing trades some of that memory for recompute by discarding selected activations during the forward pass and recomputing them as needed in the backward pass. In practice this adds roughly 30% to backward pass time but can cut peak memory usage substantially, making larger batch sizes feasible without needing a bigger GPU. Given that larger batches were strictly better on every efficiency metric we measured, this seems worth exploring.

Using idle GPU capacity rather than aiming for 100%. GPU utilization never hit 100% in any of our experiments—it peaked around 92% at batch 16 and sat at 83% for batch 4. The usual reaction to this is to view the gap as waste and try to close it. That framing makes sense for a dedicated training cluster, but on a shared resource like the Slurm cluster used here it misses an opportunity. The optimizer phase in particular is partly memory-bandwidth-bound and launches non-saturating kernels, meaning the GPU has available capacity during that window. Rather than trying to push one training job to 100% utilization, a scheduler that co-locates a lightweight secondary workload—inference requests, preprocessing for another job—during those idle windows could improve overall cluster throughput without touching the training code at all. In other words, the headroom is not necessarily a flaw in the training job; it can be a resource for the system.

Dedicated node. All runs were on shared infrastructure, which introduces noise from competing jobs. In practice the variance was small enough (under 0.6 s on run duration across three repeats) that it did not meaningfully affect our conclusions, but it would become a problem if we wanted to measure the smaller gains expected from kernel fusion or incremental batch size increases. A dedicated node would eliminate this floor on measurement uncertainty and make it much easier to detect modest improvements.

6 Discussion

Overhead is negligible. The near-zero cost of Profile C validates instrumenting every step with phase timers and NVML counters. The 1.5% overhead of Profile B is also acceptable for routine use, though its 500 ms sampling interval means per-step energy values must be inferred rather than directly measured.

The optimizer is a constant bottleneck. The 96 ms optimizer step is the most actionable finding. At batch 4 it is the single largest contributor to GPU underutilization; at batch 16 it still causes 29 recurring dips. One mitigation is to overlap the optimizer step with gradient communication in distributed training (e.g., ZeRO optimizer stages). Even in single-GPU training, fusing optimizer kernels or switching to a more GPU-friendly optimizer could reduce this cost.

Batch size and efficiency. Increasing batch size from 4 to 16 improves energy efficiency by 55% on a samples-per-joule basis. Diminishing returns between batch sizes 8 and 16 suggest the curve is flattening, and going beyond the OOM boundary would require gradient checkpointing or reduced-precision training. Since energy is proportional to run time and run time is fixed, batch size primarily affects how much useful work is done per run, not total power draw.

Limitations. All experiments use a single GPU, a single model size, and synthetic data with no tokenization or I/O overhead. The shared Slurm cluster introduces variability from competing jobs, though the small observed standard deviations ($<$ 0.6 s on duration across three repeats) suggest this was not a major issue.

7 Conclusion

Fine-grained per-step profiling of OPT-350m training is essentially free in terms of overhead. GPU utilization never saturates, with the constant 96 ms optimizer step being the primary driver of inefficiency, especially at small batch sizes where it dominates step time. Larger batches improve energy efficiency per sample processed, but the optimizer overhead remains and creates periodic utilization dips even at the maximum batch size. The most promising direction for improving efficiency is reducing or overlapping the optimizer step, either through kernel fusion or by pipelining it with other work.

References

[1] S. Zhang et al., "OPT: Open Pre-trained Transformer Language Models," *arXiv:2205.01068*, 2022.

Appendix A Resource Timelines

Figures 4–6 show the full Profile C resource timelines for each batch size. Each panel plots GPU utilization (%), GPU memory (MiB), process CPU (%), system CPU (%), per-step energy, and cumulative GPU energy over the 300-second window. Shaded bands represent one standard deviation across the three runs.

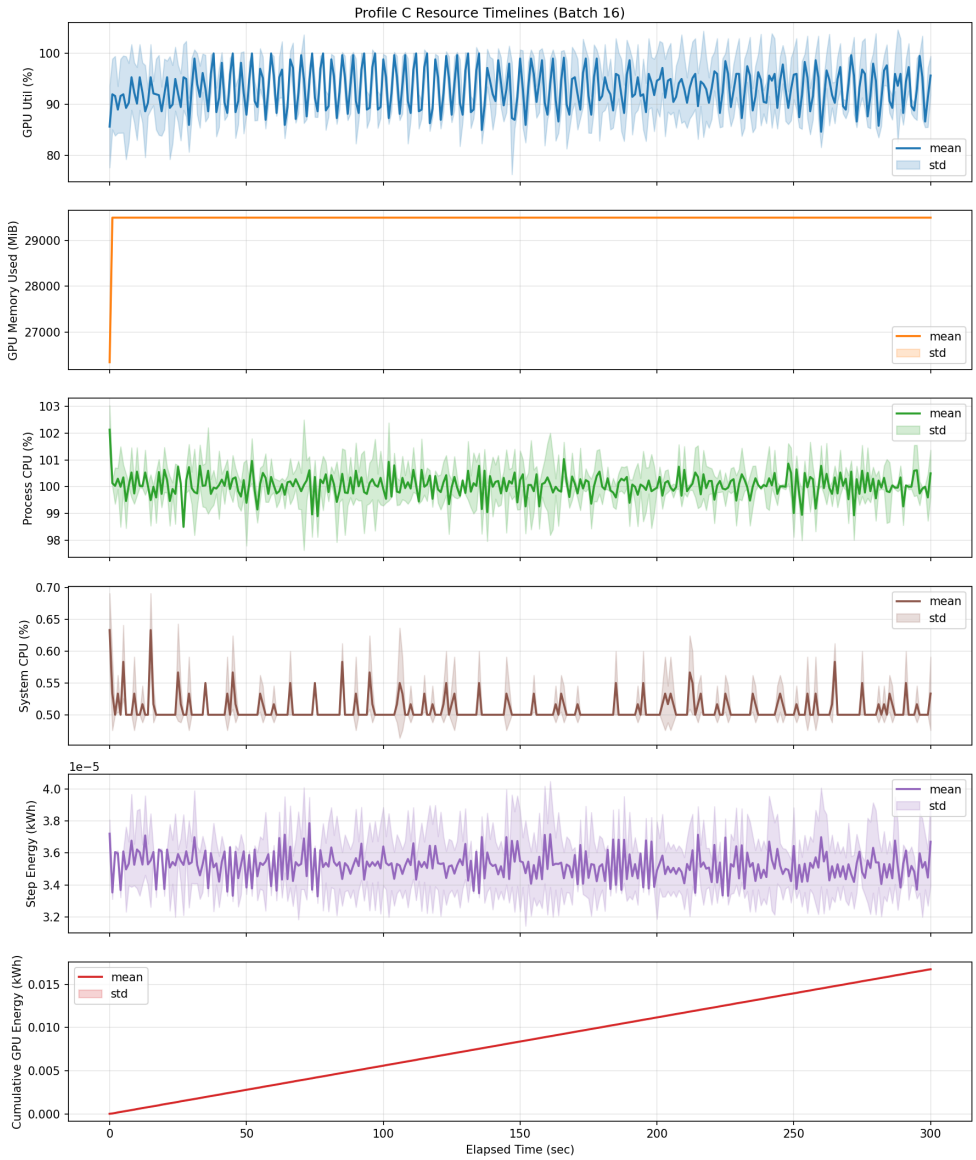


Figure 4: Resource timeline for batch size 16.

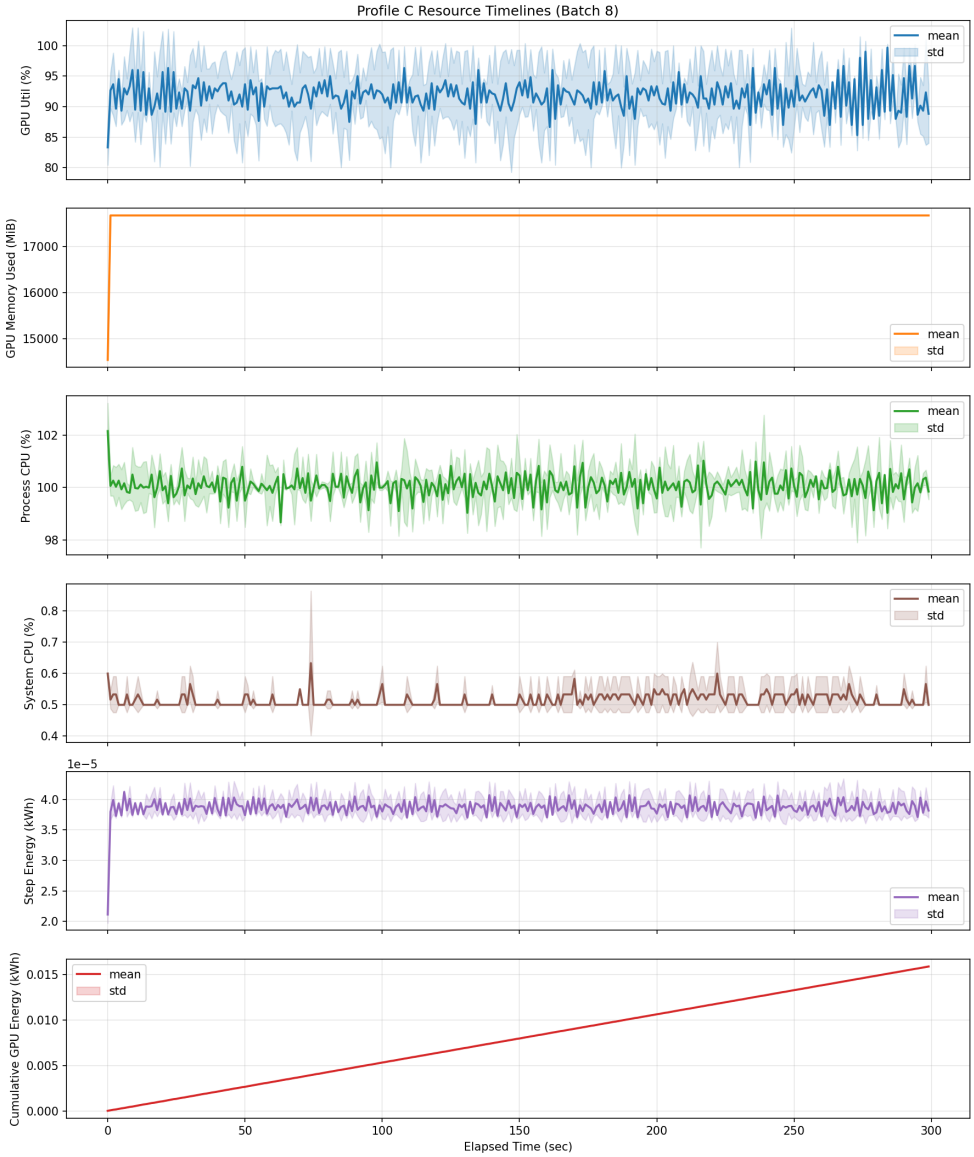


Figure 5: Resource timeline for batch size 8.

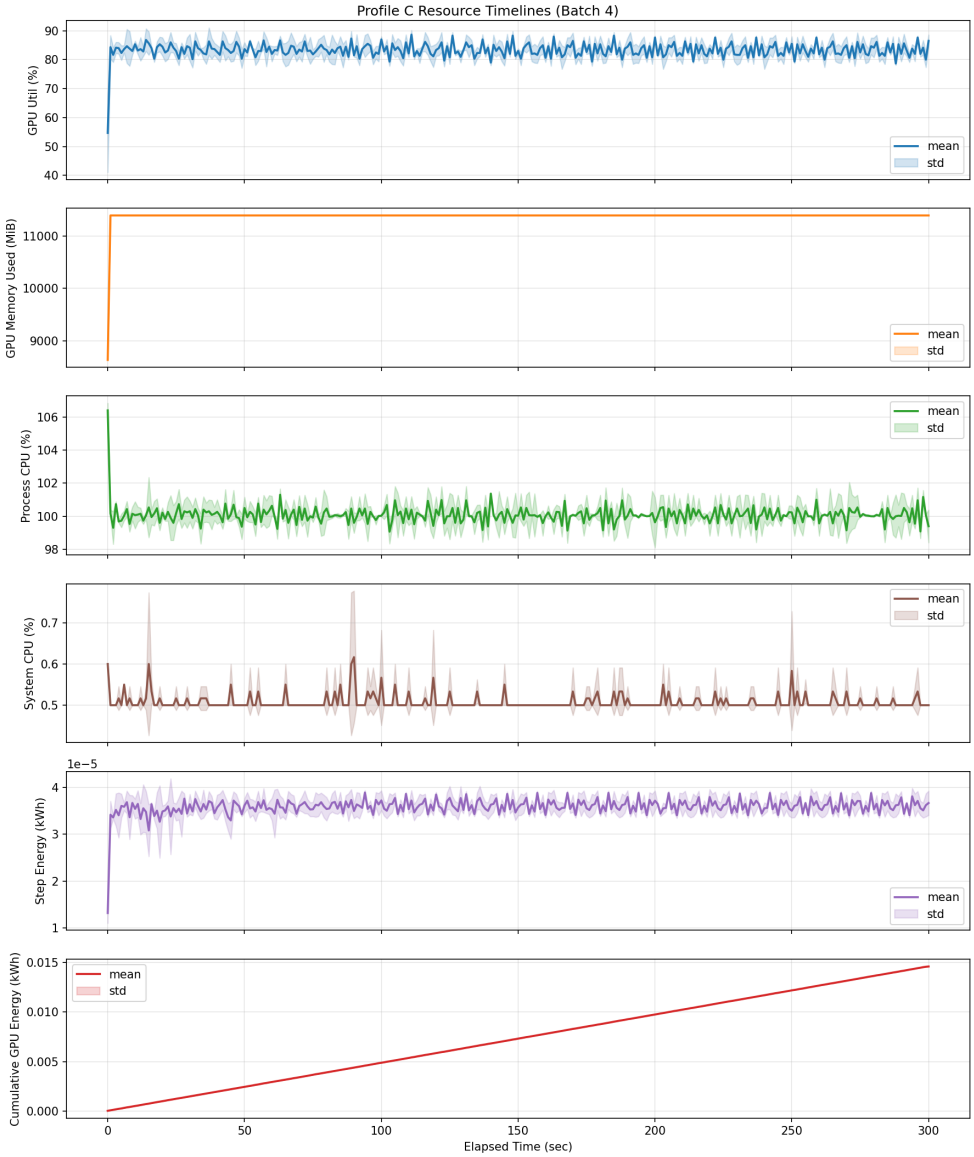
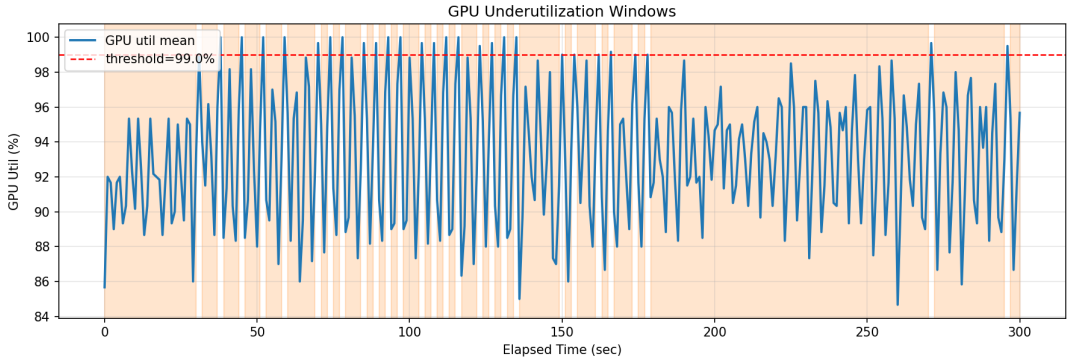
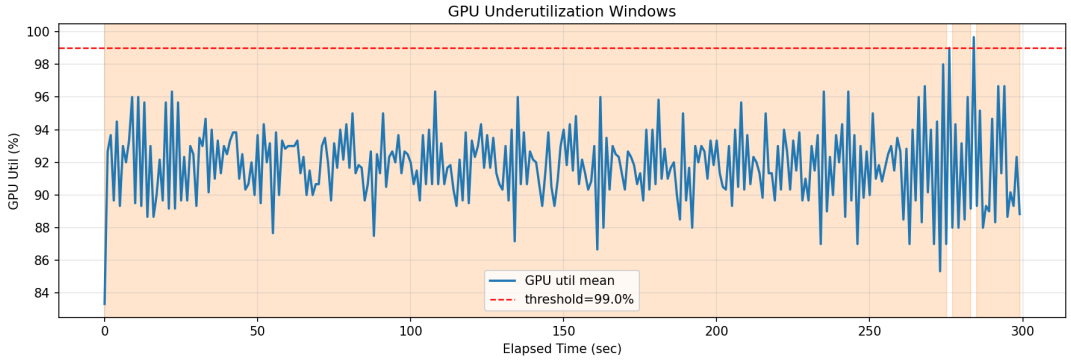


Figure 6: Resource timeline for batch size 4.

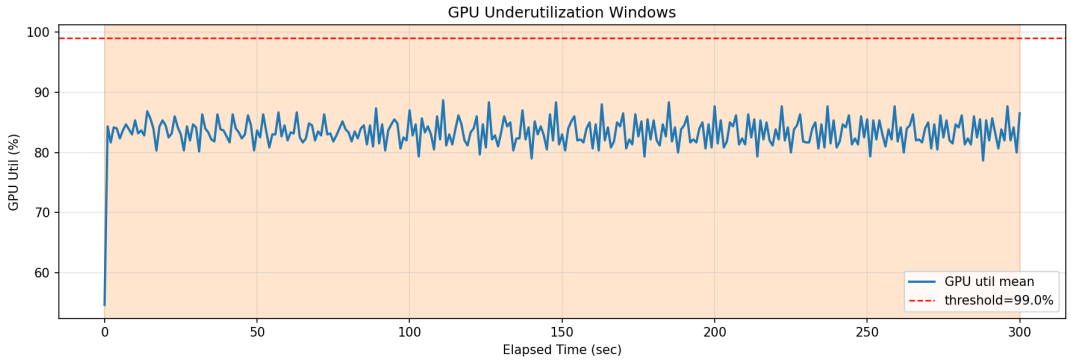
Appendix B GPU Idle Window Timelines



(a) Batch 16: 29 short dips.



(b) Batch 8: 3 windows, each roughly 7–15 seconds.



(c) Batch 4: one continuous window spanning the entire run.

Figure 7: GPU low-utilization windows (shaded) for each batch size.

Appendix C Numerical Summary

Table 3: Key metrics across batch sizes and profiles (means, 3 runs).

Batch	Profile	Duration (s)	Energy (mWh)	Steps/s	Samples/s
4	Fine-grained	309.7 ± 0.2	14.61 ± 0.01	4.05	16.2
8	Fine-grained	310.3 ± 0.6	15.90 ± 0.01	2.74	21.9
16	Fine-grained	309.7 ± 0.5	16.73 ± 0.01	1.58	25.3

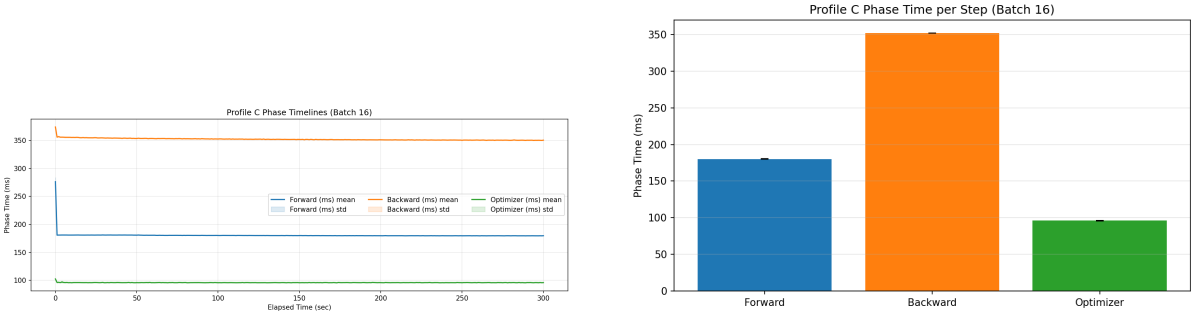
Table 4: Per-step phase timings from Profile C (means ± std, ms).

Batch	Step (ms)	Forward (ms)	Backward (ms)	Optimizer (ms)
4	245.0 ± 0.4	43.0 ± 0.0	105.3 ± 0.1	96.0 ± 0.2
8	362.8 ± 0.1	81.6 ± 0.0	184.7 ± 0.0	95.7 ± 0.1
16	629.0 ± 0.2	180.3 ± 0.1	352.1 ± 0.1	95.9 ± 0.2

Table 5: Phase composition as percentage of total step time (Profile C).

Batch	Forward (%)	Backward (%)	Optimizer (%)
4	17.5	43.0	39.2
8	22.5	50.9	26.4
16	28.7	56.0	15.2

Appendix D Phase Timeline Detail



(a) Phase timeline, batch 16.

(b) Per-step phase time, batch 16.

Figure 8: Phase timing detail for batch size 16. Durations are highly stable throughout the measurement window.